

计算机与信息工程学院实验报告

姓名	—	学号	
专业		年级	
课程	操作系统	主讲教师	刘扬
实验时间（年月日时）		2025 年 12 月 18 日	
实验地点	计算机大楼 313	辅导教师	刘扬
实验题目	文件系统与 Shell		
实验目的	通过一个简单多用户文件系统的设计，加深理解文件系统的内部功能及内部实现。		
实验环境（硬件和软件）		PC+Kali	

一、OS 课程实验

实验内容：

- 1、 可以实现下列几条命令：
 - login 用户登录
 - dir 列目录
 - create 创建文件
 - delete 删除文件
 - open 打开文件
 - close 关闭文件
 - read 读文件
 - write 写文件
- 2、 列目录时要列出文件名，物理地址，保护码和文件长度
- 3、 源文件可以进行读写保护

实验步骤：

- 1、根据实验指导书编写数据结构
- 2、编写相关函数
- 3、测试与 debug

实验数据记录：

1、

```
(kali)-(lumos@WIN-WZY)-[/mnt/_/study/操作系统/实验/实验七]
$ ./filesystem
=====
简单二级文件系统
=====
正在安装文件系统...
文件系统安装成功
===== 文件系统命令帮助 =====
login <user> <password> - 用户登录
logout - 用户注销
dir - 列出当前目录内容
create <filename> - 创建文件
delete <filename> - 删除文件
open <filename> - 打开文件
close <fd> - 关闭文件
read <fd> <size> - 读文件
write <fd> <data> - 写文件
mkdir <name> - 创建目录
chdir <path> - 切换目录
format - 格式化文件系统
install - 重新加载文件系统
halt - 关闭文件系统并退出
help - 显示帮助信息
exit - 退出系统
=====
> login 1 123456
用户 1 登录成功
> logout
用户已注销
> login 1 123456
用户 1 登录成功
> dir
===== 目录列表 =====
文件名      物理地址  保护码  文件长度
-----
.            9         rwx    0      <DIR>
..           8         rwx    0      <DIR>
=====
> create 1
文件 1 创建成功
文件关闭成功
> delete 1
文件 1 删除成功
> create 1
文件 1 创建成功
文件关闭成功
> open 1
文件 1 打开成功
文件描述符: 0
> close 0
文件关闭成功
> open 1
文件 1 打开成功
文件描述符: 0
> write 0 123123
写入 6 字节
文件关闭成功
> open 1
文件 1 打开成功
文件描述符: 0
> read 0 3
读取 3 字节: 123
> mkdir dir1
目录 dir1 创建成功
> dir
===== 目录列表 =====
文件名      物理地址  保护码  文件长度
-----
.            9         rwx    0      <DIR>
..           8         rwx    0      <DIR>
1            19        rwx    0      <DIR>
dir1         20        rwx    0      <DIR>
=====
> chdir ./dir1
> dir
===== 目录列表 =====
文件名      物理地址  保护码  文件长度
-----
.            20        rwx    0      <DIR>
..           9         rwx    0      <DIR>
=====
> format
正在格式化文件系统...
文件系统格式化完成
正在安装文件系统...
文件系统安装成功
> install
正在安装文件系统...
文件系统安装成功
> halt
正在关闭文件系统...
文件系统已关闭
程序结束
```

2、filesystem.h

```
#ifndef _FILESYS_H
#define _FILESYS_H
```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

/* 常量定义 */
#define BLOCKSIZ 512          /* 磁盘块大小 */
#define DISKSIZE 5242880     /* 磁盘总大小 5MB */
#define DISKNAME "myfsys"    /* 文件系统名称 */
#define DINODESIZ 64         /* 磁盘 i 节点大小 */
#define INODESIZ 128         /* 内存 i 节点大小 */
#define DIRSIZ 28             /* 目录名长度 */
#define PWDSIZ 14             /* 密码长度 */
#define NADDR 13              /* 地址项数量 */
#define SYSOPENFILE 40       /* 系统打开文件表大小 */
#define NOFILE 15             /* 用户打开文件表大小 */
#define NHINO 128             /* hash 表大小 */
#define USERNUM 10           /* 最大用户数 */
#define DIRNUM 32             /* 目录项数量 */
#define PWDNUM 20             /* 密码表大小 */
#define FILENUM 48            /* 文件数量 */
#define NICFREE 50            /* 空闲磁盘块栈大小 */
#define NICINOD 50           /* 空闲 i 节点数组大小 */

/* 文件类型 */
#define DEFAULTMODE 0         /* 普通文件 */
#define DIFILE 1              /* 目录文件 */
#define LARGEFILE 2          /* 大文件 */

/* 访问权限 */
#define READ 01               /* 读权限 */
#define WRITE 02              /* 写权限 */
#define EXECUTE 04            /* 执行权限 */
#define OWNER 07              /* 所有者 */

/* i 节点标志 */
#define IUPDATE 1
#define IALLOC 2

/* 文件操作标志 */
#define FREAD 1
#define FWRITE 2

```

```

/* 超级块结构 */
struct filsys {
    unsigned short s_ise;          /* i 节点块数 */
    unsigned long s_fsize;         /* 数据块数 */
    unsigned int s_nfree;          /* 空闲块数 */
    unsigned short s_pfree;        /* 空闲块指针 */
    unsigned int s_free[NICFREE];  /* 空闲块堆栈 */
    unsigned int s_ninode;         /* 空闲 i 节点数 */
    unsigned short s_pinode;       /* 空闲 i 节点指针 */
    unsigned int s_inode[NICINOD]; /* 空闲 i 节点数组 */
    unsigned int s_rinode;         /* 铭记 i 节点 */
    char s_fmod;                  /* 超级块修改标志 */
};

/* 磁盘 i 节点 */
struct dinode {
    unsigned short di_number;      /* 关联文件数 */
    unsigned short di_mode;        /* 存取权限 */
    unsigned short di_uid;         /* 用户 id */
    unsigned short di_gid;         /* 组 id */
    unsigned long di_size;         /* 文件大小 */
    unsigned int di_addr[NADDR];  /* 物理块号 */
};

/* 内存 i 节点 */
struct inode {
    struct inode *i_forw;          /* 前向指针 */
    struct inode *i_back;          /* 后向指针 */
    char i_flag;                  /* i 节点标志 */
    unsigned int i_ino;            /* 磁盘 i 节点标号 */
    unsigned int i_count;          /* 引用计数 */
    unsigned short di_number;      /* 关联文件数 */
    unsigned short di_mode;        /* 存取权限 */
    unsigned short di_uid;         /* 用户 id */
    unsigned short di_gid;         /* 组 id */
    unsigned long di_size;         /* 文件大小 */
    unsigned int di_addr[NADDR];  /* 物理块号 */
};

/* 目录项结构 */
struct direct {
    char d_name[DIRSIZ];          /* 目录名 */
    unsigned int d_ino;            /* i 节点号 */
};

```

```

/* 目录结构 */
struct dir {
    struct direct direct[DIRNUM];
    int size;
};

/* 用户密码结构 */
struct pwd {
    unsigned short p_uid;           /* 用户 id */
    unsigned short p_gid;           /* 组 id */
    char password[PWDSIZ];          /* 密码 */
};

/* 用户结构 */
struct user {
    unsigned short u_default_mode;  /* 默认权限 */
    unsigned short u_uid;           /* 用户 id */
    unsigned short u_gid;           /* 组 id */
    unsigned short u_ofile[NOFILE]; /* 用户打开文件表 */
};

/* 系统打开文件表 */
struct file {
    char f_flag;                    /* 文件操作标志 */
    unsigned int f_count;           /* 引用计数 */
    struct inode *f_inode;          /* 指向内存 i 节点 */
    unsigned long f_off;            /* 读写指针 */
};

/* hash 表结构 */
struct hinode {
    struct inode *i_forw;
};

/* 全局变量声明 */
extern struct filsys filsys;        /* 超级块 */
extern struct dir dir;              /* 目录 */
extern struct file sys_ofile[SYSOPENFILE]; /* 系统打开文件表 */
extern struct user user[USERNUM];   /* 用户表 */
extern struct pwd pwd[PWDNUM];      /* 密码表 */
extern struct hinode hinode[NHINO]; /* hash 表 */
extern struct inode *cur_path_inode; /* 当前路径 i 节点 */
extern struct inode *home_path_inode; /* 主目录 i 节点 */

```

```

extern unsigned short user_id;          /* 当前用户 id */
extern char user_name[DIRSZ];          /* 当前用户名 */
extern FILE *fd;                       /* 文件系统文件描述符 */

/* 函数声明 */

/* i 节点操作 */
struct inode *iget(unsigned int ino);
void iput(struct inode *inode);
struct inode *ialloc();
void ifree(unsigned int ino);

/* 磁盘块操作 */
unsigned int balloc();
void bfree(unsigned int block);

/* 目录和路径操作 */
struct inode *namei(char *name);
struct inode *iname(char *name);
int mkdir(char *name);
int chdir(char *path);

/* 访问控制 */
int access(struct inode *inode, int mode);

/* 文件操作 */
int create(char *filename, unsigned short mode);
int open(char *filename, int mode);
int close(int fd);
int read(int fd, char *buf, int size);
int write(int fd, char *buf, int size);
int delete(char *filename);

/* 目录显示 */
void _dir();

/* 用户管理 */
int login(char *username, char *password);
int logout();

/* 系统管理 */
int install();
void format();

```

```

void halt();

/* 辅助函数 */
unsigned int iaddr(struct inode *inode, int index);

#endif /* _FILESYS_H */

```

3、 access.c

```

#include "fileys.h"
int access(struct inode *inode, int mode)
{
    unsigned short file_mode;
    if (inode == NULL) {
        return 0;
    }
    /* 获取文件权限 */
    file_mode = inode->di_mode & 07;
    /* 检查所有者 */
    if (inode->di_uid == user[user_id].u_uid) {
        if ((file_mode & mode) == mode) {
            return 1;
        }
    }
    /* 其他用户只有读权限 */
    if (mode == READ && (file_mode & READ)) {
        return 1;
    }
    return 0;
}

```

4、 ballfre.c

```

#include "fileys.h"
unsigned int balloc()
{
    unsigned int block;
    unsigned int buf[BLOCKSIZ / sizeof(unsigned int)];
    /* 检查空闲块数 */
    if (fileys.s_nfree <= 0) {
        printf("错误: 没有空闲磁盘块\n");
        return 0;
    }
    /* 从空闲块栈中取出一个 */
    block = fileys.s_free[--fileys.s_nfree];
    /* 如果栈空,从磁盘读取下一组空闲块 */

```

```

        if (filsys.s_nfree <= 0 && block != 0) {
            fseek(fd, block * BLOCKSIZ, SEEK_SET);
            fread(buf, BLOCKSIZ, 1, fd);

            filsys.s_nfree = buf[0];
            memcpy(filsys.s_free, &buf[1], filsys.s_nfree * sizeof(unsigned
int));
        }
        /* 清空分配的块 */
        memset(buf, 0, BLOCKSIZ);
        fseek(fd, block * BLOCKSIZ, SEEK_SET);
        fwrite(buf, BLOCKSIZ, 1, fd);
        fflush(fd);
        /* 更新超级块 */
        filsys.s_fmod = 1;
        return block;
    }
    /*
    * bfree - 释放磁盘块
    * @block: 磁盘块号
    */
    void bfree(unsigned int block)
    {
        unsigned int buf[BLOCKSIZ / sizeof(unsigned int)];
        if (block == 0 || block >= filsys.s_fsize) {
            printf("错误: 无效的磁盘块号 %u\n", block);
            return;
        }
        /* 如果栈满,将当前栈写入磁盘 */
        if (filsys.s_nfree >= NICFREE) {
            buf[0] = filsys.s_nfree;
            memcpy(&buf[1], filsys.s_free, filsys.s_nfree * sizeof(unsigned
int));

            fseek(fd, block * BLOCKSIZ, SEEK_SET);
            fwrite(buf, BLOCKSIZ, 1, fd);
            fflush(fd);
            filsys.s_nfree = 0;
        }
        /* 将块号加入空闲栈 */
        filsys.s_free[filsys.s_nfree++] = block;
        filsys.s_fmod = 1;
        /* 写回超级块 */
        fseek(fd, BLOCKSIZ, SEEK_SET);

```

```
        fwrite(&filsys, sizeof(struct filsys), 1, fd);
        fflush(fd);
    }
```

5、 close.c

```
#include "filesys.h"
int close(int fd_index)
{
    int sys_fd;
    if (fd_index < 0 || fd_index >= NOFILE) {
        printf("错误: 无效的文件描述符\n");
        return -1;
    }
    sys_fd = user[user_id].u_ofile[fd_index];
    if (sys_fd == 0) {
        printf("错误: 文件未打开\n");
        return -1;
    }
    sys_fd--;
    /* 减少引用计数 */
    sys_ofile[sys_fd].f_count--;
    /* 如果引用计数为 0, 释放 i 节点 */
    if (sys_ofile[sys_fd].f_count == 0) {
        iput(sys_ofile[sys_fd].f_inode);
        sys_ofile[sys_fd].f_inode = NULL;
    }
    user[user_id].u_ofile[fd_index] = 0;
    printf("文件关闭成功\n");
    return 0;
}
```

6、 creat.c

```
#include "filesys.h"
int create(char *filename, unsigned short mode)
{
    struct inode *inode;
    struct direct direct;
    int i, fd_index;
    if (cur_path_inode == NULL) {
        printf("错误: 当前目录未设置\n");
        return -1;
    }
    /* 检查文件名长度 */
    if (strlen(filename) >= DIRSIZ) {
```

```

        printf("错误: 文件名过长\n");
        return -1;
    }
    /* 检查父目录写权限 */
    if (!access(cur_path_inode, WRITE)) {
        printf("错误: 没有写权限\n");
        return -1;
    }
    /* 检查文件是否已存在 */
    if (iname(filename) != NULL) {
        printf("错误: 文件 %s 已存在\n", filename);
        return -1;
    }
    /* 分配 i 节点 */
    inode = ialloc();
    if (inode == NULL) {
        return -1;
    }
    /* 设置文件属性 */
    inode->di_mode = mode;
    inode->di_number = 1;
    inode->di_size = 0;
    inode->i_flag |= IUPDATE;
    /* 在父目录中添加目录项 */
    strcpy(direct.d_name, filename);
    direct.d_ino = inode->i_ino;
    fseek(fd, cur_path_inode->di_addr[0] * BLOCKSIZ +
        cur_path_inode->di_size, SEEK_SET);
    fwrite(&direct, sizeof(struct direct), 1, fd);
    fflush(fd);
    cur_path_inode->di_size += sizeof(struct direct);
    cur_path_inode->i_flag |= IUPDATE;
    /* 在系统打开文件表中找空闲项 */
    for (i = 0; i < SYSOPENFILE; i++) {
        if (sys_ofile[i].f_count == 0) {
            break;
        }
    }
    if (i >= SYSOPENFILE) {
        printf("错误: 系统打开文件表已满\n");
        iput(inode);
        return -1;
    }
    /* 在用户打开文件表中找空闲项 */

```

```

        for (fd_index = 0; fd_index < NOFILE; fd_index++) {
            if (user[user_id].u_ofile[fd_index] == 0) {
                break;
            }
        }
        if (fd_index >= NOFILE) {
            printf("错误: 用户打开文件表已满\n");
            iput(inode);
            return -1;
        }
        /* 设置打开文件表项 */
        sys_ofile[i].f_flag = FREAD | FWRITE;
        sys_ofile[i].f_count = 1;
        sys_ofile[i].f_inode = inode;
        sys_ofile[i].f_off = 0;
        user[user_id].u_ofile[fd_index] = i + 1;
        printf("文件 %s 创建成功\n", filename);
        return fd_index;
    }
}

```

7、 delete.c

```

#include "filesys.h"
int delete(char *filename)
{
    struct inode *inode;
    struct direct direct;
    int i, j, found;
    if (cur_path_inode == NULL) {
        printf("错误: 当前目录未设置\n");
        return -1;
    }
    /* 检查父目录写权限 */
    if (!access(cur_path_inode, WRITE)) {
        printf("错误: 没有写权限\n");
        return -1;
    }
    /* 查找文件 */
    inode = inode(filename);
    if (inode == NULL) {
        printf("错误: 文件 %s 不存在\n", filename);
        return -1;
    }
    /* 检查是否有权限删除 */
    if (inode->di_uid != user[user_id].u_uid) {

```

```

        printf("错误: 没有删除权限\n");
        iput(inode);
        return -1;
    }
    /* 检查文件是否正在使用 */
    for (i = 0; i < SYSOPENFILE; i++) {
        if (sys_ofile[i].f_count > 0 &&
            sys_ofile[i].f_inode->i_ino == inode->i_ino) {
            printf("错误: 文件正在使用中\n");
            iput(inode);
            return -1;
        }
    }
    /* 释放数据块 */
    for (i = 0; i < 10 && inode->di_addr[i] != 0; i++) {
        bfree(inode->di_addr[i]);
    }
    /* 释放一次间接块 */
    if (inode->di_addr[10] != 0) {
        unsigned int addr[BLOCKSIZ / sizeof(unsigned int)];
        fseek(fd, inode->di_addr[10] * BLOCKSIZ, SEEK_SET);
        fread(addr, BLOCKSIZ, 1, fd);
        for (i = 0; i < BLOCKSIZ / sizeof(unsigned int) && addr[i] != 0; i++)
        {
            bfree(addr[i]);
        }
        bfree(inode->di_addr[10]);
    }
    /* 释放二次间接块 */
    if (inode->di_addr[11] != 0) {
        unsigned int addr1[BLOCKSIZ / sizeof(unsigned int)];
        unsigned int addr2[BLOCKSIZ / sizeof(unsigned int)];
        fseek(fd, inode->di_addr[11] * BLOCKSIZ, SEEK_SET);
        fread(addr1, BLOCKSIZ, 1, fd);
        for (i = 0; i < BLOCKSIZ / sizeof(unsigned int) && addr1[i] != 0; i++)
        {
            fseek(fd, addr1[i] * BLOCKSIZ, SEEK_SET);
            fread(addr2, BLOCKSIZ, 1, fd);
            for (j = 0; j < BLOCKSIZ / sizeof(unsigned int) && addr2[j] !=
0; j++) {
                bfree(addr2[j]);
            }
            bfree(addr1[i]);
        }
    }
}

```

```

        bfree(inode->di_addr[11]);
    }
    /* 释放 i 节点 */
    ifree(inode->i_ino);
    /* 从父目录中删除目录项 */
    found = -1;
    for (i = 0; i < cur_path_inode->di_size / sizeof(struct direct); i++) {
        fseek(fd, cur_path_inode->di_addr[0] * BLOCKSIZ +
            i * sizeof(struct direct), SEEK_SET);
        fread(&direct, sizeof(struct direct), 1, fd);
        if (strcmp(direct.d_name, filename) == 0) {
            found = i;
            break;
        }
    }
    if (found >= 0) {
        /* 用最后一个目录项覆盖被删除的项 */
        int last = cur_path_inode->di_size / sizeof(struct direct) - 1;
        if (found != last) {
            fseek(fd, cur_path_inode->di_addr[0] * BLOCKSIZ +
                last * sizeof(struct direct), SEEK_SET);
            fread(&direct, sizeof(struct direct), 1, fd);
            fseek(fd, cur_path_inode->di_addr[0] * BLOCKSIZ +
                found * sizeof(struct direct), SEEK_SET);
            fwrite(&direct, sizeof(struct direct), 1, fd);
        }
        cur_path_inode->di_size -= sizeof(struct direct);
        cur_path_inode->i_flag |= IUPDATE;
        fflush(fd);
    }
    iput(inode);
    printf("文件 %s 删除成功\n", filename);
    return 0;
}

```

8、dir.c

```

#include "fileys.h"
void _dir()
{
    struct direct direct;
    struct inode *inode;
    int i;
    if (cur_path_inode == NULL) {
        printf("错误: 当前目录未设置\n");
    }
}

```

```

        return;
    }
    printf("\n===== 目录列表 =====\n");
    printf("%-20s %-10s %-10s %-10s\n", "文件名", "物理地址", "保护码", "文件长度");
    printf("-----\n");
    /* 遍历目录项 */
    for (i = 0; i < cur_path_inode->di_size / sizeof(struct direct); i++) {
        fseek(fd, cur_path_inode->di_addr[0] * BLOCKSIZ +
            i * sizeof(struct direct), SEEK_SET);
        fread(&direct, sizeof(struct direct), 1, fd);
        if (direct.d_ino == 0) {
            continue;
        }
        /* 获取 i 节点 */
        inode = iget(direct.d_ino);
        if (inode == NULL) {
            continue;
        }
        /* 显示文件信息 */
        printf("%-20s ", direct.d_name);
        /* 显示物理地址 */
        if (inode->di_addr[0] != 0) {
            printf("%-10u ", inode->di_addr[0]);
        } else {
            printf("%-10s ", "N/A");
        }
        /* 显示保护码 */
        printf("%c%c%c",
            (inode->di_mode & READ) ? 'r' : '-',
            (inode->di_mode & WRITE) ? 'w' : '-',
            (inode->di_mode & EXECUTE) ? 'x' : '-');
        /* 显示文件长度 */
        if (inode->di_mode & DIFILE) {
            printf("<%s>\n", "DIR");
        } else {
            printf("%lu\n", inode->di_size);
        }
        iput(inode);
    }
    printf("=====\n\n");
}

```

```

#include "filesys.h"
struct inode *ialloc()
{
    struct inode *inode;
    unsigned int ino;
    int i;
    /* 检查空闲 i 节点数组 */
    if (filsys.s_ninode <= 0) {
        printf("错误: 没有空闲 i 节点\n");
        return NULL;
    }
    /* 从空闲 i 节点数组中取出一个 */
    ino = filsys.s_inode[--filsys.s_ninode];
    /* 获取 i 节点 */
    inode = iget(ino);
    if (inode == NULL) {
        return NULL;
    }
    /* 初始化 i 节点 */
    inode->di_number = 0;
    inode->di_mode = 0;
    inode->di_size = 0;
    inode->di_uid = user[user_id].u_uid;
    inode->di_gid = user[user_id].u_gid;
    for (i = 0; i < NADDR; i++) {
        inode->di_addr[i] = 0;
    }
    inode->i_flag |= (IUPDATE | IALLOC);
    /* 更新超级块 */
    filsys.s_fmod = 1;
    return inode;
}

void ifree(unsigned int ino)
{
    if (ino == 0 || ino > FILENUM) {
        printf("错误: 无效的 i 节点号 %u\n", ino);
        return;
    }
    /* 将 i 节点号加入空闲数组 */
    if (filsys.s_ninode >= NICINOD) {
        printf("错误: 空闲 i 节点数组已满\n");
        return;
    }
    filsys.s_inode[filsys.s_ninode++] = ino;
}

```

```

        filsys.s_fmod = 1;
        /* 写回超级块 */
        fseek(fd, BLOCKSIZ, SEEK_SET);
        fwrite(&filsys, sizeof(struct filsys), 1, fd);
        fflush(fd);
    }

```

10、igetput.c

```

#include "filesys.h"
struct hinode hinode[NHINO];
struct inode *iget(unsigned int ino)
{
    struct inode *inode, *p;
    struct dinode dinode;
    int index;
    if (ino == 0) {
        printf("错误: i 节点号为 0\n");
        return NULL;
    }
    /* 在 hash 表中查找 */
    index = ino % NHINO;
    for (inode = hinode[index].i_forw; inode != NULL; inode =
inode->i_forw) {
        if (inode->i_ino == ino) {
            inode->i_count++;
            return inode;
        }
    }
    /* 分配新的内存 i 节点 */
    inode = (struct inode *)malloc(sizeof(struct inode));
    if (inode == NULL) {
        printf("错误: 内存不足\n");
        return NULL;
    }
    /* 从磁盘读取 i 节点 */
    fseek(fd, BLOCKSIZ * 2 + (ino - 1) * DINODESIZ, SEEK_SET);
    fread(&dinode, sizeof(struct dinode), 1, fd);
    /* 初始化内存 i 节点 */
    inode->i_forw = NULL;
    inode->i_back = NULL;
    inode->i_flag = 0;
    inode->i_ino = ino;
    inode->i_count = 1;
    inode->di_number = dinode.di_number;
}

```

```

inode->di_mode = dinode.di_mode;
inode->di_uid = dinode.di_uid;
inode->di_gid = dinode.di_gid;
inode->di_size = dinode.di_size;
memcpy(inode->di_addr, dinode.di_addr, sizeof(dinode.di_addr));
/* 插入 hash 表 */
p = hinode[index].i_forw;
hinode[index].i_forw = inode;
inode->i_forw = p;
if (p != NULL) {
    p->i_back = inode;
}
inode->i_back = (struct inode *)&hinode[index];
return inode;
}
/*
 * iput - 释放内存 i 节点
 * @inode: 内存 i 节点指针
 */
void iput(struct inode *inode)
{
    struct dinode dinode;
    if (inode == NULL) {
        return;
    }
    inode->i_count--;
    if (inode->i_count > 0) {
        return;
    }
    /* 如果被修改,写回磁盘 */
    if (inode->i_flag & IUPDATE) {
        dinode.di_number = inode->di_number;
        dinode.di_mode = inode->di_mode;
        dinode.di_uid = inode->di_uid;
        dinode.di_gid = inode->di_gid;
        dinode.di_size = inode->di_size;
        memcpy(dinode.di_addr, inode->di_addr,
sizeof(dinode.di_addr));
        fseek(fd, BLOCKSIZ * 2 + (inode->i_ino - 1) * DINODESIZ,
SEEK_SET);
        fwrite(&dinode, sizeof(struct dinode), 1, fd);
        fflush(fd);
    }
    /* 从 hash 表中删除 */

```

```

        if (inode->i_back != NULL) {
            inode->i_back->i_forw = inode->i_forw;
        }
        if (inode->i_forw != NULL) {
            inode->i_forw->i_back = inode->i_back;
        }
        free(inode);
    }
    unsigned int iaddr(struct inode *inode, int index)
    {
        unsigned int addr;
        if (index < 0) {
            printf("错误: 块索引为负数\n");
            return 0;
        }
        /* 直接寻址 */
        if (index < 10) {
            return inode->di_addr[index];
        }
        /* 一次间接寻址 */
        if (index < 10 + BLOCKSIZ / sizeof(unsigned int)) {
            if (inode->di_addr[10] == 0) {
                return 0;
            }
            fseek(fd, inode->di_addr[10] * BLOCKSIZ +
                (index - 10) * sizeof(unsigned int), SEEK_SET);
            fread(&addr, sizeof(unsigned int), 1, fd);
            return addr;
        }
        /* 二次间接寻址 */
        index = index - 10 - BLOCKSIZ / sizeof(unsigned int);
        if (inode->di_addr[11] == 0) {
            return 0;
        }
        int first_level = index / (BLOCKSIZ / sizeof(unsigned int));
        int second_level = index % (BLOCKSIZ / sizeof(unsigned int));
        fseek(fd, inode->di_addr[11] * BLOCKSIZ +
            first_level * sizeof(unsigned int), SEEK_SET);
        fread(&addr, sizeof(unsigned int), 1, fd);
        if (addr == 0) {
            return 0;
        }
        fseek(fd, addr * BLOCKSIZ + second_level * sizeof(unsigned int),
            SEEK_SET);
    }

```

```
fread(&addr, sizeof(unsigned int), 1, fd);
return addr;
}
```

11、 log.c

```
#include "fileys.h"
int login(char *username, char *password)
{
    int i;
    int target_uid = -1;

    if (user_id != 0) {
        printf("错误: 已有用户登录,请先注销\n");
        return -1;
    }

    if (username != NULL && strncmp(username, "user", 4) == 0) {
        target_uid = atoi(username + 4);
    }

    /* 查找用户 */
    for (i = 0; i < PWDNUM; i++) {
        if (pwd[i].p_uid == 0) {
            continue;
        }

        if (strcmp(pwd[i].password, password) == 0 &&
            (target_uid == -1 || target_uid == pwd[i].p_uid)) {
            /* 找到匹配的密码,设置用户 ID */
            user_id = pwd[i].p_uid;
            if (username != NULL && username[0] != '\0') {
                strncpy(user_name, username, DIRSIZ - 1);
                user_name[DIRSIZ - 1] = '\0';
            } else {
                snprintf(user_name, sizeof(user_name), "user%d",
user_id);
            }

            user[user_id].u_uid = pwd[i].p_uid;
            user[user_id].u_gid = pwd[i].p_gid;
            user[user_id].u_default_mode = OWNER;

            printf("用户 %s 登录成功\n", user_name);
        }
    }
}
```

```

        /* 设置当前目录为用户主目录，如果不存在则回退到根
        目录 */
        if (cur_path_inode) { iput(cur_path_inode); cur_path_inode
= NULL; }
        if (home_path_inode) { iput(home_path_inode);
home_path_inode = NULL; }

        home_path_inode = iget(user_id + 1);
        if (home_path_inode == NULL) {
            home_path_inode = iget(1);
        }

        cur_path_inode = home_path_inode;
        if (cur_path_inode != NULL) {
            cur_path_inode->i_count++;
        }

        return 0;
    }
}

printf("错误: 用户名或密码错误\n");
return -1;
}

/*
 * logout - 注销当前用户
 */
int logout()
{
    int fd_index;

    if (user_id == 0) {
        printf("当前没有已登录用户\n");
        return 0;
    }

    /* 关闭当前用户打开的文件 */
    for (fd_index = 0; fd_index < NOFILE; fd_index++) {
        if (user[user_id].u_ofile[fd_index] != 0) {
            close(fd_index);
        }
    }
}

```

```

        if (cur_path_inode) {
            iput(cur_path_inode);
            cur_path_inode = NULL;
        }
        if (home_path_inode) {
            iput(home_path_inode);
            home_path_inode = NULL;
        }

        user_id = 0;
        user_name[0] = '\0';

        printf("用户已注销\n");
        return 0;
    }

```

12、 main.c

```

#include "fileys.h"

/* 全局变量定义 */
struct filsys filsys;
struct dir dir;
struct file sys_ofile[SYSOPENFILE];
struct user user[USERNUM];
struct pwd pwd[PWDNUM];
struct inode *cur_path_inode = NULL;
struct inode *home_path_inode = NULL;
unsigned short user_id = 0;
char user_name[DIRSIZ] = {0};
FILE *fd = NULL;

/* 函数声明 */
void help();
void command_loop();
static void init_filesystem();

void help()
{
    printf("\n===== 文件系统命令帮助 =====\n");
    printf("login <user> <password> - 用户登录\n");
    printf("logout                - 用户注销\n");
    printf("dir                    - 列出当前目录内容\n");
    printf("create <filename> - 创建文件\n");
    printf("delete <filename> - 删除文件\n");
}

```

```

printf("open <filename> - 打开文件\n");
printf("close <fd> - 关闭文件\n");
printf("read <fd> <size> - 读文件\n");
printf("write <fd> <data> - 写文件\n");
printf("mkdir <name> - 创建目录\n");
printf("chdir <path> - 切换目录\n");
printf("format - 格式化文件系统\n");
printf("install - 重新加载文件系统\n");
printf("halt - 关闭文件系统并退出\n");
printf("help - 显示帮助信息\n");
printf("exit - 退出系统\n");
printf("=====\\n\\n");
}

```

```

void command_loop()
{
    char input[256];
    char cmd[128], arg1[128], arg2[128];
    int result;

    help();

    while (1) {
        printf("> ");
        if (fgets(input, sizeof(input), stdin) == NULL) {
            break;
        }

        /* 去除换行符 */
        input[strcspn(input, "\\n")] = 0;

        /* 跳过空行 */
        if (input[0] == '\\0') {
            continue;
        }

        /* 解析命令 */
        cmd[0] = arg1[0] = arg2[0] = '\\0';
        sscanf(input, "%s %s %s", cmd, arg1, arg2);

        if (strcmp(cmd, "help") == 0) {
            help();
        }
        else if (strcmp(cmd, "login") == 0) {

```

```
        if (arg1[0] == '\0' || arg2[0] == '\0') {
            printf("用法: login <user> <password>\n");
            continue;
        }
        login(arg1, arg2);
    }
    else if (strcmp(cmd, "logout") == 0) {
        logout();
    }
    else if (strcmp(cmd, "dir") == 0) {
        if (user_id == 0) {
            printf("错误: 请先登录\n");
            continue;
        }
        _dir();
    }
    else if (strcmp(cmd, "create") == 0) {
        if (user_id == 0) {
            printf("错误: 请先登录\n");
            continue;
        }
        if (arg1[0] == '\0') {
            printf("用法: create <filename>\n");
            continue;
        }
        result = create(arg1, OWNER);
        if (result >= 0) {
            close(result);
        }
    }
    else if (strcmp(cmd, "delete") == 0) {
        if (user_id == 0) {
            printf("错误: 请先登录\n");
            continue;
        }
        if (arg1[0] == '\0') {
            printf("用法: delete <filename>\n");
            continue;
        }
        delete(arg1);
    }
    else if (strcmp(cmd, "open") == 0) {
        if (user_id == 0) {
            printf("错误: 请先登录\n");
```

```

        continue;
    }
    if (arg1[0] == '\0') {
        printf("用法: open <filename>\n");
        continue;
    }
    result = open(arg1, FREAD | FWRITE);
    if (result >= 0) {
        printf("文件描述符: %d\n", result);
    }
}
else if (strcmp(cmd, "close") == 0) {
    if (user_id == 0) {
        printf("错误: 请先登录\n");
        continue;
    }
    if (arg1[0] == '\0') {
        printf("用法: close <fd>\n");
        continue;
    }
    close(atoi(arg1));
}
else if (strcmp(cmd, "read") == 0) {
    if (user_id == 0) {
        printf("错误: 请先登录\n");
        continue;
    }
    if (arg1[0] == '\0' || arg2[0] == '\0') {
        printf("用法: read <fd> <size>\n");
        continue;
    }
    int fd_idx = atoi(arg1);
    int size = atoi(arg2);
    char *buffer = (char *)malloc(size + 1);
    if (buffer) {
        int n = read(fd_idx, buffer, size);
        if (n > 0) {
            buffer[n] = '\0';
            printf("读取 %d 字节: %s\n", n, buffer);
        } else if (n == 0) {
            printf("已到文件末尾, 没有数据可读\n");
        } else {
            printf("读取失败\n");
        }
    }
}

```

```

        free(buffer);
    } else {
        printf("错误: 内存分配失败\n");
    }
}
else if (strcmp(cmd, "write") == 0) {
    if (user_id == 0) {
        printf("错误: 请先登录\n");
        continue;
    }
    if (arg1[0] == '\0' || arg2[0] == '\0') {
        printf("用法: write <fd> <data>\n");
        continue;
    }
    int fd_idx = atoi(arg1);
    int n = write(fd_idx, arg2, strlen(arg2));
    if (n > 0) {
        printf("写入 %d 字节\n", n);
    }
}
else if (strcmp(cmd, "mkdir") == 0) {
    if (user_id == 0) {
        printf("错误: 请先登录\n");
        continue;
    }
    if (arg1[0] == '\0') {
        printf("用法: mkdir <name>\n");
        continue;
    }
    mkdir(arg1);
}
else if (strcmp(cmd, "chdir") == 0) {
    if (user_id == 0) {
        printf("错误: 请先登录\n");
        continue;
    }
    if (arg1[0] == '\0') {
        printf("用法: chdir <path>\n");
        continue;
    }
    chdir(arg1);
}
else if (strcmp(cmd, "format") == 0) {
    format();
}

```

```

        install();
    }
    else if (strcmp(cmd, "install") == 0) {
        install();
    }
    else if (strcmp(cmd, "halt") == 0) {
        halt();
        break;
    }
    else if (strcmp(cmd, "exit") == 0) {
        halt();
        break;
    }
    else {
        printf("未知命令: %s\n", cmd);
        printf("输入 help 查看帮助信息\n");
    }
}

}

int main(int argc, char *argv[])
{
    printf("\n===== \n");
    printf("    简单二级文件系统\n");
    printf("===== \n\n");

    init_filesystem();
    command_loop();

    if (fd != NULL) {
        halt();
    }

    printf("\n 程序结束\n");
    return 0;
}

static void init_filesystem()
{
    /* 若安装失败则尝试格式化再安装 */
    if (install() != 0) {
        format();
        if (install() != 0) {
            printf("错误: 无法安装文件系统\n");

```

```

        exit(1);
    }
}

```

13、 open.c

```

#include "fileys.h"
int open(char *filename, int mode)
{
    struct inode *inode;
    int i, fd_index;
    if (cur_path_inode == NULL) {
        printf("错误: 当前目录未设置\n");
        return -1;
    }
    /* 查找文件 */
    inode = iname(filename);
    if (inode == NULL) {
        printf("错误: 文件 %s 不存在\n", filename);
        return -1;
    }
    /* 检查访问权限 */
    if (mode & FREAD) {
        if (!access(inode, READ)) {
            printf("错误: 没有读权限\n");
            iput(inode);
            return -1;
        }
    }
    if (mode & FWRITE) {
        if (!access(inode, WRITE)) {
            printf("错误: 没有写权限\n");
            iput(inode);
            return -1;
        }
    }
    /* 在系统打开文件表中查找 */
    for (i = 0; i < SYSOPENFILE; i++) {
        if (sys_ofile[i].f_count > 0 &&
            sys_ofile[i].f_inode->i_ino == inode->i_ino) {
            /* 文件已打开,增加引用计数 */
            sys_ofile[i].f_count++;
            break;
        }
    }
}

```

```

    }
    /* 文件未打开,分配新表项 */
    if (i >= SYSOPENFILE) {
        for (i = 0; i < SYSOPENFILE; i++) {
            if (sys_ofile[i].f_count == 0) {
                break;
            }

            if (i >= SYSOPENFILE) {
                printf("错误: 系统打开文件表已满\n");
                iput(inode);
                return -1;
            }
            sys_ofile[i].f_flag = mode;
            sys_ofile[i].f_count = 1;
            sys_ofile[i].f_inode = inode;
            sys_ofile[i].f_off = 0;
        } else {
            iput(inode); /* 已在系统表中,释放新获取的 i 节点 */
        }
        /* 在用户打开文件表中找空闲项 */
        for (fd_index = 0; fd_index < NOFILE; fd_index++) {
            if (user[user_id].u_ofile[fd_index] == 0) {
                break;
            }
        }
        if (fd_index >= NOFILE) {
            printf("错误: 用户打开文件表已满\n");
            sys_ofile[i].f_count--;
            if (sys_ofile[i].f_count == 0) {
                iput(sys_ofile[i].f_inode);
            }
            return -1;
        }
        user[user_id].u_ofile[fd_index] = i + 1;
        printf("文件 %s 打开成功\n", filename);
        return fd_index;
    }
}

```

14、rdwt.c

```

#include "filesys.h"
int read(int fd_index, char *buf, int size)
{
    int sys_fd;

```

```

struct file *file;
struct inode *inode;
unsigned int block, offset, bytes_to_read; //bytes_read,
char block_buf[BLOCKSIZE];
int total_read = 0;
if (fd_index < 0 || fd_index >= NOFILE) {
    printf("错误: 无效的文件描述符\n");
    return -1;
}
sys_fd = user[user_id].u_ofile[fd_index];
if (sys_fd == 0) {
    printf("错误: 文件未打开\n");
    return -1;
}
sys_fd--;
file = &sys_ofile[sys_fd];
inode = file->f_inode;
/* 检查读权限 */
if (!(file->f_flag & FREAD)) {
    printf("错误: 文件不可读\n");
    return -1;
}
/* 检查读取范围 */
if (file->f_off >= inode->di_size) {
    return 0; /* 已到文件末尾 */
}
if (file->f_off + size > inode->di_size) {
    size = inode->di_size - file->f_off;
}
/* 逐块读取 */
while (total_read < size) {
    /* 计算当前块号和偏移 */
    block = iaddr(inode, file->f_off / BLOCKSIZE);
    if (block == 0) {
        break;
    }
    offset = file->f_off % BLOCKSIZE;
    bytes_to_read = BLOCKSIZE - offset;
    if (bytes_to_read > size - total_read) {
        bytes_to_read = size - total_read;
    }
    /* 读取数据块 */
    fseek(fd, block * BLOCKSIZE, SEEK_SET);
    fread(block_buf, BLOCKSIZE, 1, fd);
}

```

```

        memcpy(buf + total_read, block_buf + offset, bytes_to_read);
        total_read += bytes_to_read;
        file->f_off += bytes_to_read;
    }
    return total_read;
}

int write(int fd_index, char *buf, int size)
{
    int sys_fd;
    struct file *file;
    struct inode *inode;
    unsigned int block, offset, bytes_to_write; //bytes_written,
    char block_buf[BLOCKSIZ];
    int total_written = 0;
    if (fd_index < 0 || fd_index >= NOFILE) {
        printf("错误: 无效的文件描述符\n");
        return -1;
    }
    sys_fd = user[user_id].u_ofile[fd_index];
    if (sys_fd == 0) {
        printf("错误: 文件未打开\n");
        return -1;
    }
    sys_fd--;
    file = &sys_ofile[sys_fd];
    inode = file->f_inode;
    /* 检查写权限 */
    if (!(file->f_flag & FWRITE)) {
        printf("错误: 文件不可写\n");
        return -1;
    }
    /* 逐块写入 */
    while (total_written < size) {
        /* 计算当前块号和偏移 */
        int block_index = file->f_off / BLOCKSIZ;
        block = iaddr(inode, block_index);
        /* 如果块不存在,分配新块 */
        if (block == 0) {
            block = balloc();
            if (block == 0) {
                printf("错误: 磁盘空间不足\n");
                break;
            }
        }
        /* 更新 i 节点地址 */
    }
}

```

```

        if (block_index < 10) {
            inode->di_addr[block_index] = block;
        } else if (block_index < 10 + BLOCKSIZ / sizeof(unsigned int)) {
            /* 一次间接寻址 */
            if (inode->di_addr[10] == 0) {
                inode->di_addr[10] = balloc();
                if (inode->di_addr[10] == 0) {
                    bfree(block);
                    break;
                }
            }
            fseek(fd, inode->di_addr[10] * BLOCKSIZ +
                (block_index - 10) * sizeof(unsigned int), SEEK_SET);
            fwrite(&block, sizeof(unsigned int), 1, fd);
        } else {
            /* 二次间接寻址 */
            int first_level = (block_index - 10 - BLOCKSIZ / sizeof(unsigned
int)) /
                (BLOCKSIZ / sizeof(unsigned int));
            int second_level = (block_index - 10 - BLOCKSIZ /
sizeof(unsigned int)) %
                (BLOCKSIZ / sizeof(unsigned int));
            unsigned int addr;
            if (inode->di_addr[11] == 0) {
                inode->di_addr[11] = balloc();
                if (inode->di_addr[11] == 0) {
                    bfree(block);
                    break;
                }
            }
            fseek(fd, inode->di_addr[11] * BLOCKSIZ +
                first_level * sizeof(unsigned int), SEEK_SET);
            fread(&addr, sizeof(unsigned int), 1, fd);
            if (addr == 0) {
                addr = balloc();
                if (addr == 0) {
                    bfree(block);
                    break;
                }
            }
            fseek(fd, inode->di_addr[11] * BLOCKSIZ +
                first_level * sizeof(unsigned int), SEEK_SET);
            fwrite(&addr, sizeof(unsigned int), 1, fd);
        }
        fseek(fd, addr * BLOCKSIZ + second_level * sizeof(unsigned

```

```

int), SEEK_SET);

        fwrite(&block, sizeof(unsigned int), 1, fd);
    }
    inode->i_flag |= IUPDATE;
}
offset = file->f_off % BLOCKSIZ;
bytes_to_write = BLOCKSIZ - offset;
if (bytes_to_write > size - total_written) {
    bytes_to_write = size - total_written;
}
/* 如果不是完整块写入,先读取原数据 */
if (offset != 0 || bytes_to_write < BLOCKSIZ) {
    fseek(fd, block * BLOCKSIZ, SEEK_SET);
    fread(block_buf, BLOCKSIZ, 1, fd);
}
/* 复制数据到块缓冲 */
memcpy(block_buf + offset, buf + total_written, bytes_to_write);
/* 写入数据块 */
fseek(fd, block * BLOCKSIZ, SEEK_SET);
fwrite(block_buf, BLOCKSIZ, 1, fd);
fflush(fd);
total_written += bytes_to_write;
file->f_off += bytes_to_write;
/* 更新文件大小 */
if (file->f_off > inode->di_size) {
    inode->di_size = file->f_off;
    inode->i_flag |= IUPDATE;
}
}
return total_written;
}

```

15、format.c

```

#include "fileys.h"
void format()
{
    int i, dir_entries;
    struct dinode dinode;
    struct direct root_dir[DIRNUM];
    //unsigned int block;
    char buf[BLOCKSIZ];
    int inode_blocks;
    char username[DIRSIZ];
}

```

```

printf("正在格式化文件系统...\n");

/* 创建文件系统文件 */
fd = fopen(DISKNAME, "wb+");
if (fd == NULL) {
    printf("错误: 无法创建文件系统文件\n");
    return;
}

/* 初始化超级块 */
memset(&filsys, 0, sizeof(struct filsys));
filsys.s_ismode = FILENUM;
filsys.s_fsize = DISKSIZE / BLOCKSIZ;
filsys.s_nfree = 0;
filsys.s_ninode = 0;
filsys.s_fmod = 0;

/* 计算 i 节点区大小(块数) */
inode_blocks = (FILENUM * DINODESIZ + BLOCKSIZ - 1) / BLOCKSIZ;

/* 初始化空闲 i 节点数组 */
for (i = FILENUM; i > USERNUM + 1; i--) {
    if (filsys.s_ninode < NICINOD) {
        filsys.s_inode[filsys.s_ninode++] = i;
    }
}

/* 初始化空闲磁盘块栈 */
for (i = filsys.s_fsize - 1; i >= 2 + inode_blocks + USERNUM + 1; i--) {
    if (filsys.s_nfree < NICFREE) {
        filsys.s_free[filsys.s_nfree++] = i;
    } else {
        /* 栈满,写入磁盘 */
        memset(buf, 0, BLOCKSIZ);
        *(unsigned int *)buf = filsys.s_nfree;
        memcpy(buf + sizeof(unsigned int), filsys.s_free,
            filsys.s_nfree * sizeof(unsigned int));

        fseek(fd, i * BLOCKSIZ, SEEK_SET);
        fwrite(buf, BLOCKSIZ, 1, fd);

        filsys.s_nfree = 0;
    }
}

```

```

/* 写入引导块(块 0) */
memset(buf, 0, BLOCKSIZ);
strcpy(buf, "Simple File System - Boot Block");
fseek(fd, 0, SEEK_SET);
fwrite(buf, BLOCKSIZ, 1, fd);

/* 初始化 i 节点区 */
memset(&dinode, 0, sizeof(struct dinode));
fseek(fd, BLOCKSIZ * 2, SEEK_SET);
for (i = 0; i < FILENUM; i++) {
    fwrite(&dinode, sizeof(struct dinode), 1, fd);
}

/* 创建根目录 i 节点(i 节点 1) */
dinode.di_number = 1;
dinode.di_mode = DIFILE | OWNER;
dinode.di_uid = 0;
dinode.di_gid = 0;
dinode.di_addr[0] = 2 + inode_blocks;

/* 创建根目录内容, 附带用户主目录入口 */
dir_entries = 0;
strcpy(root_dir[dir_entries].d_name, ".");
root_dir[dir_entries++].d_ino = 1;
strcpy(root_dir[dir_entries].d_name, "..");
root_dir[dir_entries++].d_ino = 1;

for (i = 1; i <= USERNUM && dir_entries < DIRNUM; i++) {
    snprintf(username, sizeof(username), "user%d", i);
    strcpy(root_dir[dir_entries].d_name, username);
    root_dir[dir_entries++].d_ino = i + 1;
}

dinode.di_size = dir_entries * sizeof(struct direct);

fseek(fd, BLOCKSIZ * 2, SEEK_SET);
fwrite(&dinode, sizeof(struct dinode), 1, fd);

fseek(fd, dinode.di_addr[0] * BLOCKSIZ, SEEK_SET);
fwrite(root_dir, sizeof(struct direct), dir_entries, fd);

/* 创建用户主目录 */
for (i = 1; i <= USERNUM; i++) {

```

```
dinode.di_number = 1;
dinode.di_mode = DIFILE | OWNER;
dinode.di_uid = i;
dinode.di_gid = i;
dinode.di_size = 2 * sizeof(struct direct);
dinode.di_addr[0] = 2 + inode_blocks + i;

fseek(fd, BLOCKSIZ * 2 + i * DINODESIZ, SEEK_SET);
fwrite(&dinode, sizeof(struct dinode), 1, fd);

/* 创建用户目录内容 */
strcpy(root_dir[0].d_name, ".");
root_dir[0].d_ino = i + 1;
strcpy(root_dir[1].d_name, "..");
root_dir[1].d_ino = 1;

fseek(fd, dinode.di_addr[0] * BLOCKSIZ, SEEK_SET);
fwrite(root_dir, sizeof(struct direct), 2, fd);
}

/* 初始化密码表 */
for (i = 0; i < PWDNUM; i++) {
    pwd[i].p_uid = 0;
    pwd[i].p_gid = 0;
    pwd[i].password[0] = '\0';
}

/* 添加默认用户 */
pwd[0].p_uid = 1;
pwd[0].p_gid = 1;
strcpy(pwd[0].password, "123456");

pwd[1].p_uid = 2;
pwd[1].p_gid = 2;
strcpy(pwd[1].password, "passwd");

/* 将最新的超级块写回磁盘 */
fseek(fd, BLOCKSIZ, SEEK_SET);
fwrite(&filsys, sizeof(struct filsys), 1, fd);
fflush(fd);
fclose(fd);
fd = NULL;

printf("文件系统格式化完成\n");
```

```
}
```

16、 install.c

```
#include "fileys.h"

int install()
{
    int i;

    printf("正在安装文件系统...\n");

    /* 打开文件系统文件 */
    fd = fopen(DISKNAME, "rb+");
    if (fd == NULL) {
        printf("错误: 无法打开文件系统文件,请先格式化\n");
        return -1;
    }

    /* 读取超级块 */
    fseek(fd, BLOCKSIZ, SEEK_SET);
    fread(&filsys, sizeof(struct filsys), 1, fd);

    /* 初始化 hash 表 */
    for (i = 0; i < NHINO; i++) {
        hinode[i].i_forw = NULL;
    }

    /* 初始化系统打开文件表 */
    for (i = 0; i < SYSOPENFILE; i++) {
        sys_ofile[i].f_flag = 0;
        sys_ofile[i].f_count = 0;
        sys_ofile[i].f_inode = NULL;
        sys_ofile[i].f_off = 0;
    }

    /* 初始化用户表 */
    for (i = 0; i < USERNUM; i++) {
        user[i].u_uid = i + 1;
        user[i].u_gid = i + 1;
        user[i].u_default_mode = OWNER;
        memset(user[i].u_ofile, 0, sizeof(user[i].u_ofile));
    }

    /* 初始化密码表（内存中维护） */
    for (i = 0; i < PWDNUM; i++) {
```

```

        pwd[i].p_uid = 0;
        pwd[i].p_gid = 0;
        pwd[i].password[0] = '\0';
    }
    pwd[0].p_uid = 1; pwd[0].p_gid = 1; strcpy(pwd[0].password,
"123456");
    pwd[1].p_uid = 2; pwd[1].p_gid = 2; strcpy(pwd[1].password,
"passwd");

    /* 设置当前用户和路径 */
    user_id = 0;
    user_name[0] = '\0';

    if (cur_path_inode) { iput(cur_path_inode); cur_path_inode = NULL; }
    if (home_path_inode) { iput(home_path_inode); home_path_inode =
NULL; }

    cur_path_inode = iget(1);
    if (cur_path_inode != NULL) {
        home_path_inode = cur_path_inode;
        home_path_inode->i_count++;
    }

    printf("文件系统安装成功\n");
    return 0;
}

```

17、 name.c

```

#include "fileys.h"

struct inode *namei(char *name)
{
    struct inode *inode, *parent_inode;
    struct direct direct;
    char filename[DIRSIZ];
    char *p;
    int i, found;

    /* 空路径名 */
    if (name == NULL || name[0] == '\0') {
        return NULL;
    }

    /* 绝对路径从根目录开始,相对路径从当前目录开始 */
    if (name[0] == '/') {

```

```

        inode = iget(1); /* 根目录 i 节点号为 1 */
        name++;
    } else {
        if (cur_path_inode == NULL) {
            return NULL;
        }
        inode = cur_path_inode;
        inode->i_count++;
    }

    /* 逐级查找 */
    while (*name != '\0') {
        /* 提取文件名 */
        p = filename;
        while (*name != '/' && *name != '\0' && p - filename < DIRSIZ - 1)
        {
            *p++ = *name++;
        }
        *p = '\0';

        if (*name == '/') {
            name++;
        }

        /* 检查是否为目录 */
        if ((inode->di_mode & DIFILE) == 0) {
            printf("错误: %s 不是目录\n", filename);
            iput(inode);
            return NULL;
        }

        /* 在目录中查找 */
        found = 0;
        for (i = 0; i < inode->di_size / sizeof(struct direct); i++) {
            fseek(fd, inode->di_addr[0] * BLOCKSIZ +
                i * sizeof(struct direct), SEEK_SET);
            fread(&direct, sizeof(struct direct), 1, fd);

            if (strcmp(direct.d_name, filename) == 0) {
                parent_inode = inode;
                inode = iget(direct.d_ino);
                iput(parent_inode);
                found = 1;
                break;
            }
        }
    }

```

```

        }
    }

    if (!found) {
        printf("错误: 找不到文件 %s\n", filename);
        iput(inode);
        return NULL;
    }
}

return inode;
}

struct inode *iname(char *name)
{
    struct inode *inode;
    struct direct direct;
    int i;

    if (cur_path_inode == NULL) {
        printf("错误: 当前目录未设置\n");
        return NULL;
    }

    /* 在当前目录中查找 */
    for (i = 0; i < cur_path_inode->di_size / sizeof(struct direct); i++) {
        fseek(fd, cur_path_inode->di_addr[0] * BLOCKSIZ +
            i * sizeof(struct direct), SEEK_SET);
        fread(&direct, sizeof(struct direct), 1, fd);

        if (strcmp(direct.d_name, name) == 0) {
            inode = iget(direct.d_ino);
            return inode;
        }
    }

    return NULL;
}

int mkdir(char *name)
{
    struct inode *inode;
    struct direct dirent;
    struct direct newdir[2];

```

```
unsigned int block;

if (cur_path_inode == NULL) {
    printf("错误: 当前目录未设置\n");
    return -1;
}

if (name == NULL || name[0] == '\0') {
    printf("错误: 目录名不能为空\n");
    return -1;
}

if (strlen(name) >= DIRSIZ) {
    printf("错误: 目录名过长\n");
    return -1;
}

if (!access(cur_path_inode, WRITE)) {
    printf("错误: 没有写权限\n");
    return -1;
}

inode = inode(name);
if (inode != NULL) {
    printf("错误: %s 已存在\n", name);
    iput(inode);
    return -1;
}

inode = ialloc();
if (inode == NULL) {
    return -1;
}

block = balloc();
if (block == 0) {
    iput(inode);
    return -1;
}

inode->di_mode = DIFILE | OWNER;
inode->di_number = 1;
inode->di_size = 2 * sizeof(struct direct);
inode->di_addr[0] = block;
```

```

inode->i_flag |= IUPDATE;

strcpy(newdir[0].d_name, ".");
newdir[0].d_ino = inode->i_ino;
strcpy(newdir[1].d_name, "..");
newdir[1].d_ino = cur_path_inode->i_ino;

fseek(fd, block * BLOCKSIZ, SEEK_SET);
fwrite(newdir, sizeof(struct direct), 2, fd);
fflush(fd);

strcpy(dirent.d_name, name);
dirent.d_ino = inode->i_ino;

fseek(fd, cur_path_inode->di_addr[0] * BLOCKSIZ +
cur_path_inode->di_size, SEEK_SET);
fwrite(&dirent, sizeof(struct direct), 1, fd);
fflush(fd);

cur_path_inode->di_size += sizeof(struct direct);
cur_path_inode->i_flag |= IUPDATE;

printf("目录 %s 创建成功\n", name);

iput(inode);
return 0;
}

int chdir(char *path)
{
    struct inode *inode;
    struct inode *old;

    if (path == NULL || path[0] == '\0') {
        printf("错误: 请输入路径\n");
        return -1;
    }

    inode = namei(path);
    if (inode == NULL) {
        return -1;
    }

    if ((inode->di_mode & DIFILE) == 0) {

```

```

        printf("错误: 目标不是目录\n");
        iput(inode);
        return -1;
    }

    if (!access(inode, READ)) {
        printf("错误: 没有进入目录的权限\n");
        iput(inode);
        return -1;
    }

    old = cur_path_inode;
    cur_path_inode = inode;

    if (old) {
        iput(old);
    }

    return 0;
}

```

问题讨论:

- 1、 程序会在当前目录创建名为 `mysys` 的文件作为虚拟磁盘，要先进行格式化。
- 2、 文件系统大小设置为 5MB。
- 3、 首次运行会自动格式化文件系统。
- 4、 权限控制时 `access()` 使用当前用户与文件 `di_uid` 检查; 开/写/删前都先校验; 删除时确认无其他打开引用。
- 5、 在 `iget/iput` 配对, `namei/chdir`、`open/close`、`logout/halt` 中, 需要仔细处理 `i_count` 否则可能会造成泄漏或过度释放等错误。
- 6、 在设计目录时, 目录的大小应为目录项数 $\times$ `sizeof(struct direct)`, 并且写入目录时要检查写权限